MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión A del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_lopez_abraham
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/lopez_abraham/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
¿Qué diferencia hay entre una clase y un objeto? Usa la idea de plano e instancia.

Tu respuesta

```
una clase es la base en la cual se define algo, principalmente caracteristicas o
comportamientos, es decir es un plano o plantilla, mientras que el objeto es la instancia
creada a partir del plano.
```

Corrección. Bien: la clase es el plano o plantilla que define características y comportamientos, y el objeto es la instancia creada a partir de ella. Faltó decir que cada objeto tiene estado propio, es decir, sus propios valores de los atributos de instancia.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué es la firma de un método y qué partes la forman? Explica si incluye el tipo de retorno.

Tu respuesta

```
la frima de un metodo es el identificador del copilador, para ser distinguido dentro de una
clase. Esta formado por el nombre del metodo y su lista de parametros.
```

Corrección. Bien: la firma la forman el nombre del método y sus parámetros, y sirve para distinguirlo dentro de la clase. Faltó precisar que cuentan los tipos de los parámetros en orden y contestar lo que se pedía explícitamente: el tipo de retorno no forma parte de la firma.

Pregunta 0.3 · Preguntas abiertas

Pregunta

Compara la anatomía de una variable local y un atributo de instancia: dónde se declaran, valor inicial y cuánto viven.

Tu respuesta

```
V.L(donde se declaran):dentro de un metodo constructor o bloque
V.L(valor inicial): se declara fuera de la clase
V.L(cuanto viven): solo existen cuando se ejecuta el bloque
A.I(donde se declaran ): se declaran dentro de la clase
A.I(valor inicial): tienen valores por defecto
A.I(cuanto viven):todo el tiempo que este dentro de la memoria
```

Corrección. Bien: la local se declara en un método, constructor o bloque y existe solo mientras ese bloque se ejecuta; el atributo se declara en la clase, recibe un valor por defecto y vive mientras existe el objeto. Error: para el valor inicial de la local escribiste "se declara fuera de la clase"; lo correcto es que no tiene valor por defecto y debe asignarse antes de leerse.

Pregunta 0.4 · Preguntas abiertas

Pregunta

¿Qué significa null en una referencia? Explica qué ocurre al llamar un método sobre esa referencia.

Tu respuesta

```
generalmente null en una referencia guarda una direccion de memoria, pero ya no me acuerdo
bien que ocurre al llamar un metodo sobre este.
```

Corrección. Incorrecto: null no guarda una dirección de memoria, indica que la referencia no apunta a ningún objeto. Además, llamar un método de instancia sobre una referencia null hace que el programa truene en ejecución (NullPointerException); asignar null no falla.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿Cuándo conviene usar for-each para recorrer un arreglo y cuándo necesitas un recorrido con índice?

Tu respuesta

```
para recorrrer un arreglo conviene
```

Corrección. La respuesta quedó incompleta ("para recorrer un arreglo conviene") y no tiene una idea evaluable. Se esperaba: for-each cuando solo necesitas cada valor (por ejemplo, para sumar), y for con índice cuando necesitas la posición o reemplazar elementos.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
StringBuilder ruta = new StringBuilder("S");
StringBuilder alias = ruta;
for (int i = 1; i <= 3; i++) {
    alias.append(i / 2);
    System.out.println(ruta);
}
String texto = "S" + 2 + 1;
System.out.println(texto);
double mitad = (double) (7 / 2);
System.out.println(mitad);
```

Tu respuesta

```
System.out.println(ruta);
dezconozco que imprima ("S" 21/2)
System.out.println(texto);
imprime "S" 21
System.out.println(mitad);
desconozco que imprima
```

Corrección. Para la variable texto escribiste "S" 21: el contenido S21 es correcto (concatenación de izquierda a derecha), pero las comillas y el espacio no aparecen en la salida. No diste las tres líneas del ciclo ni la última. La salida correcta es S0, S01, S011, S21 y 3.0: alias y ruta son el mismo StringBuilder, y 7/2 es división entera antes del cast, por eso da 3.0.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
double medida = 6.8;
int unidades = medida;
System.out.println(unidades);
```

Tu respuesta

Clasificación

```
Compila pero truena
```

Causa o salida

```
System.out.println(mitad);
truena porque son terminos diferentes
```

Corrección. La clasificación es incorrecta: no compila. La causa que diste se refiere a la variable mitad de otro reactivo y "son términos diferentes" no es una causa concreta. Lo correcto: no hay conversión implícita de double a int, hace falta un cast explícito.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int[] canales = {2, 5, 8};
int indice = canales.length;
System.out.println(canales[indice]);
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
System.out.println(canales[indice]);
si funciona, la salida debe ser 2, he de suponer
```

Corrección. Incorrecto: compila pero truena. length vale 3 y el último índice válido es 2, así que canales[3] lanza ArrayIndexOutOfBoundsException y no imprime nada.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
String codigo = new String("F2");
System.out.println(codigo == "F2");
System.out.println(codigo.equals("F2"));
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
String codigo = new String("F2");
System.out.println(codigo == "F2");
```

```
no se puede comparar cadenas con ==
```

Corrección. Incorrecto: sí compila y funciona. Comparar cadenas con == está permitido, solo compara identidad (si son el mismo objeto). Por eso imprime false y luego true, porque equals compara el contenido.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada multímetro conserva su inventario y nace sin responsable. Se cuenta el total de prestados. Escribe prestar(persona) y una consulta del responsable. Supón que solo se llama prestar con un aparato disponible y un nombre válido.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿La cuenta de prestados pertenece a cada aparato o a todos? ¿Qué dato nunca cambia?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

```
class Multimetro {
int persona;
int


}
```

Justificaciones

```
___
```

Corrección. Solo declaraste un atributo de instancia (int persona), sin justificar; el responsable debería ser String. Faltaron el atributo final de inventario, el contador static de prestados, el constructor, los métodos prestar y la consulta del responsable, la respuesta al reto y las justificaciones. Reto: la cuenta de prestados es de todos los aparatos (static) y el inventario es el dato que nunca cambia (final).

R4 · Código con repetición

Pregunta

Un equipo guarda sus folios como enteros no negativos. Devuelve cuántos dígitos tiene folio usando while y división entre 10. El folio 0 tiene un dígito. No conviertas el número a texto.

```
static int contarDigitos(int folio)
```

Ejemplos (argumentos → retorno):

4072 → 4 · 0 → 1 · 8 → 1

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
static int contarDigitos(int folio)
```

Corrección. Solo escribiste la cabecera, que está bien, pero el método no tiene cuerpo. Faltó un while que divida folio entre 10 y cuente una vez por división, con el caso de 0 devolviendo 1 y un return del contador.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Clasifica la carga de un equipo. Usa switch clásico sobre carga / 10. Puedes validar antes con if. Devuelve el texto de la tabla. Los límites están incluidos. No imprimas ni leas del teclado.

```
static String nivelCarga(int carga)
```

Regla o valor recibido	Texto devuelto
Menor que 0 o mayor que 50	"INVALIDA"
0 a 19	"BAJA"
20 a 39	"MEDIA"
40 a 50	"ALTA"

Tu respuesta

```
static String nivelCarga(int carga){
  if ( carga <0)
  System.out.println("INVALIDA");
  else (carga >50)
  System.out.printl("INVALIDA");
  switch (modo){
   case A: System.out.printl("BAJA")
   case B: System.out.printl("MEDIA")
   case c: System.out.printl("ALTA")
  if case A (carga = 0)
  System.out.printl("BAJA");
  else case A (carga >19)
  System.out.printl("BAJA");



}
```

Corrección. Bien: la cabecera, y la idea de validar carga < 0 y carga > 50 como INVALIDA antes del switch; la tres etiquetas BAJA/MEDIA/ALTA aparecen en los case. Errores: el switch es sobre modo (no existe) en vez de carga / 10; no agrupa los cocientes 0/1, 2/3 y 4/5; no hay ningún return; imprime aunque el enunciado lo prohíbe; y la sintaxis está rota (printl, faltan punto y coma y llaves, "if case A"). Lo correcto era validar con if y return "INVALIDA", y luego switch (carga / 10) con case 0: case 1: return "BAJA"; case 2: case 3: return "MEDIA"; default: return "ALTA".

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.